

Polymorphism in Java

A Comprehensive Guide

Slide 1: Introduction

What is Polymorphism in Java?

- Polymorphism means "**many forms**" — the same method call can produce different behavior
- Java supports **compile-time polymorphism** (method overloading) and **runtime polymorphism** (method overriding)
- Enables writing flexible, reusable code that works with multiple types
- Core principle: treat objects by their **parent type**, behavior determined by **actual type**

```
Animal a = new Dog();  
a.speak();    // Calls Dog's speak() – not Animal's
```

Slide 2: Why Polymorphism Matters

Benefits:

- **Flexibility** — Code that works with the base type also works with all subtypes
- **Extensibility** — Add new subclasses without changing existing code
- **Maintainability** — Reduces conditional logic (switch/if-else on type)
- **Abstraction** — Focus on what objects DO, not what they ARE

```
Without polymorphism:           With polymorphism:  
if (a instanceof Dog)           a.speak();    // Works for any subclass  
    ((Dog)a).bark();  
else if (a instanceof Cat)  
    ((Cat)a).meow();
```

Slide 3: Two Types of Polymorphism

Aspect	Compile-Time (Overloading)	Runtime (Overriding)
When resolved	At compilation	At runtime
Also called	Static binding, early binding	Dynamic binding, late binding
Based on	Method signature (name + params)	Object type (not reference)
Methods	Can be static or instance	Must be instance method

Aspect	Compile-Time (Overloading)	Runtime (Overriding)
Performance	Faster (no lookup)	Slight overhead (vtable lookup)

```
// Compile-time: which print() is called determined by argument types
class Printer {
    void print(int x) { }
    void print(String x) { }
    void print(double x) { }
}

// Runtime: which speak() runs determined by actual object
Animal a = new Dog();
a.speak();    // Dog's speak() at runtime
```

Slide 4: Method Overloading — Compile-Time Polymorphism

Same method name, different parameters:

```
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }

    public double add(double a, double b) {
        return a + b;
    }
}

Calculator calc = new Calculator();
System.out.println(calc.add(2, 3));           // 5
System.out.println(calc.add(2, 3, 4));        // 9
System.out.println(calc.add(2.5, 3.5));       // 6.0
```

- Resolved at **compile time** based on argument types and count
- Return type alone is **not** sufficient for overloading

Slide 5: Method Overloading Rules

Valid overloading requires different parameter lists:

```
class Display {
    // Valid overloads – different parameter types
    void show(int x) { }
    void show(double x) { }

    // Valid overloads – different parameter count
    void show(int x, int y) { }

    // Valid overloads – different parameter order
    void show(String s, int x) { }
    void show(int x, String s) { }

    // INVALID: same name, same params, different return type
    // int show(int x) { return x; }      // Compile error
    // void show(int x) { }              // Already defined

    // INVALID: varargs vs array (same effective signature)
    // void display(int... x) { }
    // void display(int[] x) { }          // Compile error
}
```

Slide 6: Method Overriding — Runtime Polymorphism

Redefining a parent method in the child:

```
class Animal {
    void speak() {
        System.out.println("Animal speaks");
    }
}

class Dog extends Animal {
    @Override
    void speak() {
        System.out.println("Woof!");
    }

    void fetch() {
        System.out.println("Fetching...");
    }
}

class Cat extends Animal {
    @Override
    void speak() {
        System.out.println("Meow!");
    }
}
```

Same method signature, different implementation — behavior determined at **runtime** by the actual object type.

Slide 7: Polymorphic Method Invocation

```
Animal a1 = new Animal();
Animal a2 = new Dog();
Animal a3 = new Cat();

a1.speak();    // Animal speaks
a2.speak();    // Woof!           ← runtime polymorphism
a3.speak();    // Meow!           ← runtime polymorphism

// Compile-time type: Animal (reference)
// Runtime type:      Dog, Cat (actual object)
// Method resolved by: runtime type
```

Key insight: The compiler checks the reference type (`Animal`), but the JVM calls the method on the actual object type (`Dog`, `Cat`).

Slide 8: The `@Override` Annotation

Why use it?

```
class Parent {
    void display() { }
}

class Child extends Parent {
    @Override
    void display() { }           // Compiler: confirming override

    // @Override
    // void displayY() { }       // COMPILE ERROR: no such method in Parent
}
```

- Not required, but **strongly recommended**
 - Compiler checks the method actually overrides something
 - Catches typos: `disply()` vs `display()`
 - Documents intent for other developers
-

Slide 9: Overriding Rules

A method must satisfy ALL conditions to override:

1. Same **name**
2. Same **parameter list** (order and types)
3. Same **return type** or **covariant return type** (subclass of original)
4. Same or **wider** access modifier (not narrower)
5. Cannot be **private**, **static**, or **final**

```
class Parent {  
    protected Number getValue() { return 10; }  
}  
  
class Child extends Parent {  
    @Override  
    public Integer getValue() { return 20; }  
    // OK: protected → public (wider)  
    // OK: Number → Integer (covariant)  
}
```

Slide 10: Covariant Return Type

Returning a more specific type in the override:

```
class Shape {  
    Shape clone() {  
        return new Shape();  
    }  
}  
  
class Circle extends Shape {  
    @Override  
    Circle clone() {                // Returns Circle, not Shape  
        return new Circle();  
    }  
}  
  
// Before Java 5:  
@Override  
Shape clone() { return new Circle(); } // Must return Shape  
// Caller needed explicit downcast  
  
// Java 5+:  
Circle c = new Circle().clone();      // No cast needed!
```

- Introduced in Java 5 (JDK 1.5)
- Eliminates unnecessary downcasting
- Type-safe — compiler ensures the return type is a subclass

Slide 11: Polymorphism with Arrays

Polymorphic containers:

```
Animal[] animals = new Animal[3];
animals[0] = new Dog();
animals[1] = new Cat();
animals[2] = new Dog();

for (Animal a : animals) {
    a.speak();    // Correct override called for each
}
// Output:
// Woof!
// Meow!
// Woof!
```

Array declared as `Animal[]` but holds `Dog` and `Cat` objects. Iterating polymorphically — each calls its own `speak()`.

Slide 12: Polymorphism with Method Parameters

Writing methods that accept any subtype:

```
class AnimalShelter {
    void adopt(Animal a) {           // Accept any Animal subclass
        System.out.print("Adopting: ");
        a.speak();
    }
}

AnimalShelter shelter = new AnimalShelter();
shelter.adopt(new Dog());           // Adopting: Woof!
shelter.adopt(new Cat());           // Adopting: Meow!
shelter.adopt(new Animal());        // Adopting: Animal speaks
```

- `adopt(Animal a)` works with `Animal` and all its subclasses
 - Adding a new `Bird` class requires zero changes to `adopt()`
 - This is the **Open/Closed Principle**: open for extension, closed for modification
-

Slide 13: Polymorphism with Collections

```
import java.util.*;

class Zoo {
```

```
private List<Animal> animals = new ArrayList<>();

void addAnimal(Animal a) {
    animals.add(a);
}

void makeAllSpeak() {
    for (Animal a : animals) {
        a.speak();
    }
}

int count() {
    return animals.size();
}
}

Zoo zoo = new Zoo();
zoo.addAnimal(new Dog());
zoo.addAnimal(new Cat());
zoo.addAnimal(new Dog());
zoo.makeAllSpeak();
// Woof! Meow! Woof!
```

Slide 14: Polymorphism and instanceof

Checking runtime type:

```
class Animal {
    void speak() { System.out.println("Animal speaks"); }
}

class Dog extends Animal {
    @Override
    void speak() { System.out.println("Woof!"); }

    void fetch() { System.out.println("Fetching..."); } // Dog-only
}

Animal a = new Dog();

System.out.println(a instanceof Animal); // true
System.out.println(a instanceof Dog);    // true

// Safe downcasting with instanceof to access subclass-specific method
if (a instanceof Dog) {
    Dog d = (Dog) a;           // Safe cast
    d.fetch();                 // Only accessible after downcast
}
```

```
// Avoid: checking types defeats polymorphism
// Better: move the behavior into the class itself
```

When to use instanceof:

- Framework code (serialization, cloning)
- When the supertype cannot be modified
- Rare cases where behavior fundamentally differs

When to avoid: When the behavior can be added as a method in the base class.

Slide 15: Polymorphism with Abstract Classes

Defining a polymorphic contract:

```
abstract class Shape {
    protected String color;

    Shape(String color) {
        this.color = color;
    }

    abstract double area();           // Must be overridden
    abstract double perimeter();      // Must be overridden

    String getColor() {               // Concrete – inherited
        return color;
    }
}

class Circle extends Shape {
    private double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double area() { return Math.PI * radius * radius; }

    @Override
    double perimeter() { return 2 * Math.PI * radius; }
}
```

Slide 16: Polymorphic Array with Abstract Type

```
class Rectangle extends Shape {
    private double width, height;

    Rectangle(String color, double w, double h) {
        super(color);
        this.width = w;
        this.height = h;
    }

    @Override
    double area() { return width * height; }

    @Override
    double perimeter() { return 2 * (width + height); }
}

// Polymorphic usage
Shape[] shapes = {
    new Circle("red", 5),
    new Rectangle("blue", 4, 6)
};

for (Shape s : shapes) {
    System.out.printf("%s: area=%.2f, perimeter=%.2f\n",
        s.getColor(), s.area(), s.perimeter());
}

// Cannot: Shape s = new Shape("green"); // COMPILE ERROR
```

Slide 17: Polymorphism with Interfaces

Pure polymorphic contracts:

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {
    @Override
    public void fly() {
        System.out.println("Duck flying");
    }

    @Override
    public void swim() {
```

```
        System.out.println("Duck swimming");
    }
}

class Airplane implements Flyable {
    @Override
    public void fly() {
        System.out.println("Airplane flying");
    }
}

// Polymorphic: treat as Flyable
Flyable[] flyers = {new Duck(), new Airplane()};
for (Flyable f : flyers) {
    f.fly();
}
```

Slide 18: Interfaces — Multiple Inheritance of Behavior

Java classes can implement MULTIPLE interfaces:

```
interface Printable {
    void print();
}

interface Saveable {
    void save();
}

interface Exportable {
    void export(String format);
}

class Document implements Printable, Saveable, Exportable {
    @Override
    public void print() {
        System.out.println("Printing document");
    }

    @Override
    public void save() {
        System.out.println("Saving document");
    }

    @Override
    public void export(String format) {
        System.out.println("Exporting as " + format);
    }
}
```

```
// Can be used as any of the interface types
Printable p = new Document();
Saveable s = new Document();
Exportable e = new Document();
```

Slide 19: Default Methods in Interfaces (Java 8+)

Adding behavior to interfaces without breaking implementations:

```
interface Vehicle {
    void start();

    default void honk() {                // Default implementation
        System.out.println("Beep!");
    }

    static boolean isMoving(Vehicle v) { // Static utility
        return true; // Simplified
    }
}

class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car starting");
    }
    // honk() inherited with default implementation
}

Car c = new Car();
c.start();           // Car starting
c.honk();            // Beep! – from default method
Vehicle.isMoving(c); // true – static interface method
```

Slide 20: Default Method Resolution with Polymorphism

```
interface A {
    default void show() {
        System.out.println("A.show");
    }
}

interface B {
    default void show() {
        System.out.println("B.show");
    }
}
```

```
// Class implementing both – MUST resolve conflict
class Child implements A, B {
    @Override
    public void show() {
        A.super.show();    // Explicitly call A's version
        B.super.show();    // Explicitly call B's version
        System.out.println("Child.show");
    }
}

// Polymorphic dispatch
A ref1 = new Child();
ref1.show();    // A.show / B.show / Child.show

B ref2 = new Child();
ref2.show();    // A.show / B.show / Child.show – same result
```

Slide 21: The Diamond Problem in Java

Java forces explicit resolution:

```

      Vehicle (interface)
     /         \
  Car           Boat   (interfaces with default show())
   \           /
    Amphibious (class)

```

```
interface Car { default void show() { System.out.println("Car"); } }
interface Boat { default void show() { System.out.println("Boat"); } }

class Amphibious implements Car, Boat {
    @Override
    public void show() {
        Car.super.show();    // Must pick or combine
        Boat.super.show();
    }
}
```

- Compiler requires override — no ambiguity
- Diamond problem is **resolved**, not avoided

Slide 22: Polymorphism with **super** Keyword

Calling the overridden parent method:

```
class Employee {
    protected String name;

    Employee(String name) {
        this.name = name;
    }

    String getDetails() {
        return "Employee: " + name;
    }
}

class Manager extends Employee {
    private int teamSize;

    Manager(String name, int teamSize) {
        super(name);          // Call parent constructor
        this.teamSize = teamSize;
    }

    @Override
    String getDetails() {
        return super.getDetails() + ", manages " + teamSize + " people";
    }
}

Manager m = new Manager("Alice", 5);
System.out.println(m.getDetails());
// Employee: Alice, manages 5 people
```

Slide 23: Polymorphism with Equality (`equals()`)

Overriding `equals` polymorphically:

```
import java.util.Objects;

class Person {
    private String name;
    private int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
```

```

        Person other = (Person) obj;
        return age == other.age && Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }
}

// Polymorphic equals – Object parameter accepts anything
Person p1 = new Person("Alice", 30);
Person p2 = new Person("Alice", 30);
System.out.println(p1.equals(p2));    // true
System.out.println(p1.equals("Alice")); // false (different class)

```

Slide 24: The Template Method Pattern

Polymorphism to define algorithm skeleton:

```

abstract class DataProcessor {
    // Template method – defines the algorithm structure
    public final void process() {
        String data = readData();
        String transformed = transform(data);
        String result = analyze(transformed);
        output(result);
    }

    private String readData() {
        return "raw data";    // Common step
    }

    protected abstract String transform(String data);
    protected abstract String analyze(String data);

    private void output(String result) {
        System.out.println("Result: " + result);    // Common step
    }
}

class ReverseProcessor extends DataProcessor {
    @Override protected String transform(String data) {
        return new StringBuilder(data).reverse().toString();
    }

    @Override protected String analyze(String data) {
        return "Reversed: " + data;
    }
}

```

```
}  
}
```

Slide 25: The Strategy Pattern

Polymorphism to swap algorithms:

```
interface SortStrategy {  
    void sort(int[] data);  
}  
  
class BubbleSort implements SortStrategy {  
    @Override  
    public void sort(int[] data) {  
        System.out.println("Bubble sort");  
    }  
}  
  
class QuickSort implements SortStrategy {  
    @Override  
    public void sort(int[] data) {  
        System.out.println("Quick sort");  
    }  
}  
  
class Sorter {  
    private SortStrategy strategy;  
  
    Sorter(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    void setStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    void sortData(int[] data) {  
        strategy.sort(data);    // Polymorphic dispatch  
    }  
}  
  
int[] data = {3, 1, 4, 1, 5};  
Sorter s = new Sorter(new QuickSort());  
s.sortData(data);    // Quick sort  
  
s.setStrategy(new BubbleSort());  
s.sortData(data);    // Bubble sort
```

Slide 26: Polymorphism vs Overloading — Side by Side

```
class Example {
    // Overloading – compile time, same class
    void process(int x) {
        System.out.println("int: " + x);
    }

    void process(String x) {
        System.out.println("String: " + x);
    }

    // Overriding – runtime, subclass
    void handle() {
        System.out.println("Example.handle");
    }
}

class SubExample extends Example {
    @Override
    void handle() {
        System.out.println("SubExample.handle");
    }
}

Example e = new SubExample();
e.process(42);           // int: 42 – compile time (overloading)
e.process("Hi");        // String: Hi – compile time (overloading)
e.handle();             // SubExample.handle – runtime (overriding)
```

Slide 27: Common Mistake — Confusing Overload and Override

```
class Parent {
    void show(int x) {
        System.out.println("Parent: " + x);
    }
}

class Child extends Parent {
    // INTENDED: override show(int)
    // ACTUALLY: overload – different parameter type!
    void show(Integer x) {           // Integer ≠ int
        System.out.println("Child: " + x);
    }
}

Child c = new Child();
c.show(42);                       // Parent: 42 ← not Child's version!
// int → Integer requires autoboxing, but overload resolution
```

```
// picks the exact match (int → int) over autoboxed version

// Correct override:
@Override
void show(int x) { ... } // Same signature as parent
```

Slide 28: Common Mistake — Calling Overridable Method from Constructor

```
class Parent {
    Parent() {
        init(); // Calls overridden method!
    }

    void init() {
        System.out.println("Parent.init");
    }
}

class Child extends Parent {
    private String name = "Alice";

    Child() {
        super(); // init() called here!
        System.out.println("Child constructor");
    }

    @Override
    void init() {
        System.out.println("Child.init: " + name);
        // name is null at this point! (before assignment)
    }
}

new Child();
// Output:
// Child.init: null ← name not yet initialized!
// Child constructor
```

Slide 29: Common Mistake — Static Methods Are NOT Polymorphic

```
class Parent {
    static void whoAmI() {
        System.out.println("Parent");
    }
}
```

```
class Child extends Parent {
    // HIDES, not overrides
    static void whoAmI() {
        System.out.println("Child");
    }
}

Parent p = new Child();
p.whoAmI();           // "Parent" – static, NOT polymorphic!

// Resolution based on REFERENCE type, not object type
// Static methods belong to the CLASS, not the instance
```

Rule: Use `ClassName.staticMethod()` syntax. Don't call static methods on instances — it's misleading.

Slide 30: Common Mistake — Covariant Return with Primitives

```
class Parent {
    int getValue() { return 10; }
}

class Child extends Parent {
    // INVALID: int is not a reference type
    // @Override
    // long getValue() { return 20; }    // COMPILE ERROR

    // Also invalid – different return type
    // @Override
    // double getValue() { return 20.0; } // COMPILE ERROR
}
```

Covariant return types only work with **reference types** (objects), not primitives. Primitive return types must match exactly.

Slide 31: Best Practices

Do's:

- ✓ Program to interfaces / abstract types, not concrete classes
- ✓ Use `@Override` on every overriding method
- ✓ Design for polymorphism or seal/final your classes
- ✓ Keep method contracts consistent (Liskov Substitution Principle)
- ✓ Use abstract classes for shared state, interfaces for behavior contracts
- ✓ Favor composition over inheritance for code reuse

Don'ts:

- X Call overridable methods from constructors
 - X Use `instanceof` checks instead of polymorphic dispatch
 - X Confuse overloading (compile time) with overriding (runtime)
 - X Make methods `final` unnecessarily (prevents extension)
 - X Create deep inheritance hierarchies (more than 3 levels)
-

Slide 32: Summary

Polymorphism in Java:

Type	When	Mechanism	Example
Compile-time	At compilation	Overloading	<code>print(int)</code> vs <code>print(String)</code>
Runtime	At execution	Overriding	<code>Animal a = new Dog(); a.speak()</code>

Key Mechanisms:

- **Method Overriding** — `@Override`, same signature, different body
- **Abstract Classes** — `abstract` methods force override in subclasses
- **Interfaces** — `implements` multiple types, `default` methods
- **Covariant Returns** — Return subclass of original return type
- **`super` Keyword** — Call parent's overridden method or constructor

Core Principle: Treat objects as their **most general type** (parent class or interface) and let the **runtime** determine which method implementation to invoke. This makes code flexible, testable, and extensible.

Slide 33: Resources & Further Reading

Official Documentation:

- Java Polymorphism: docs.oracle.com/javase/tutorial/java/landl/polymorphism.html
- Overriding: docs.oracle.com/javase/tutorial/java/landl/override.html
- Interfaces: docs.oracle.com/javase/tutorial/java/landl/createinterface.html
- Default Methods: docs.oracle.com/javase/tutorial/java/landl/defaultmethods.html

Topics for Further Study:

- Generics and wildcards (`? extends T`, `? super T`) for polymorphic containers
 - Sealed classes and interfaces (Java 17+)
 - Records and pattern matching for type-safe polymorphism
 - Visitor pattern — externalizing polymorphic behavior
 - Dependency injection for runtime strategy selection
-

Thank You!

Questions?